

Data Pipeline: Architectural Flow Prod

1. Introduction:

This document outlines the ideal architecture for building a scalable, efficient, and reliable data pipeline to ingest, preprocess, aggregate, and deliver insights from Hive-partitioned CSV data stored in an S3-compatible UpCloud bucket.

The objective is to ensure dynamic ingestion of new data, handle frequent historical changes gracefully, and produce insights such as summations and averages while maintaining extensibility for future enhancements like streaming ingestion, data cataloging, monitoring, and full data lineage.

2. Problem Statment:

- The dataset is organized as CSV files in an S3-like storage with Hive-style partitioning (year/month/day).
- New files are added dynamically.
- Files for specific dates may change frequently within the same day.
- Insights need to be dynamically updated with minimal manual intervention.
- Bonus objectives include optional streaming ingestion, data lineage, cataloging, monitoring, and full idempotency.

3. Proposed Architecture Overview:

 Please find the architectural diagram on this board.

4. Component Description:

Component	Description
Upcloud S3 bucket	Source and destination storage for raw and processed data.
Apache Airflow	Orchestrator for ingestion, preprocessing, aggregation, monitoring, and insight generation. Includes alerting mechanisms.
Integrated Ingestion +Preprocessing	Executed by AWS Lambda functions for lightweight operations, or by Kubernetes Pods for long-running tasks. Python scripts handle data extraction, schema validation, and preprocessing. In our case we assume we can do it through a lambda function
Apache Iceberg	Data Lakehouse table format ensuring schema evolution, ACID operations, partition evolution, and time-travel.
dbt core	Clean, modular, and maintainable SQL-based transformation framework, isolated and reproducible in Docker containers. These models will be configured to use trino as a sql query engine.

Trino	Fast, scalable SQL engine optimized for object storage queries.
Slack	Real-time error notifications to engineering team.
Grafana	Visual dashboard monitoring for operational metrics.
Terraform	Ensures reproducibility and consistency when provisioning Trino catalogs and table structures, avoiding manual setup errors.

5. Detailed Data Flow

6. **File Detection:** I am going to do the orchestration of the pipeline through Airflow. But there are two ways we can do this depending on the needs of the data.

a. Streaming Ingestion:

- i. The S3-compatible bucket is configured with event notifications (e.g., via webhook, SNS/SQS equivalent).
- ii. When a file is created or modified, it triggers a Lambda function or webhook.
- iii. This handler then calls Airflow's REST API (`/dags/{dag_id}/dagRuns`) to trigger a DAG run with the relevant partition metadata.

b. Batch Ingestion:

- i. We can schedule the DAG periodically to run x times a day until the end of day to run the pipeline for the specific partition (in our case date). We will need to be sensible about the timings and when the file for the last date changes. For example running a pipeline for reading a file from partition 2025/05/01 can be scheduled at 3AM, then 12 PM, 18PM and 23:59 PM but we would need to be sure that the file for this date doesn't change after this time. In all cases, schedule can be a bit more dynamic. This solution will mimic a batch processing.

7. **Ingestion + Preprocessing:**

- Upon file detection, a task loads the new/changed CSVs.
- **Schema Validation** is performed before ingestion to ensure that the incoming data matches the expected schema (column names, types, nullability).
- Basic preprocessing includes type casting, deduplication, and missing value handling.
- This step is handled by AWS Lambda functions for lightweight files. If the processing time or data volume exceeds Lambda limits, Kubernetes Pods will be used.
- Preprocessed data is written into the Raw Zone as Iceberg tables partitioned by `year`, `month`, and `day`.

8. **Aggregation:**

- a. dbt Core models, running inside Docker containers triggered by Airflow, incrementally process only the new or modified partitions.

- b. Data validation tests are run pre- and post-aggregation:
 - i. **Pre-DBT Run:** Validate schema, row counts, basic null checks.
 - ii. **Post-DBT Run:** Validate aggregates, uniqueness, and partition integrity.
 - c. Aggregated tables (e.g., daily billing summaries) are generated and stored in the Aggregated Zone.
- 9. **Insight Serving:**
 - a. Trino queries the Aggregated Zone tables to deliver insights (e.g., total billing per day, average usage).
 - b. Terraform scripts manage the creation of Trino tables and ensure proper schema and partition definitions are consistently maintained across raw and aggregated zones.
- 10. **Monitoring and Alerting:**
 - a. Airflow task failures automatically trigger Slack alerts.
 - b. Grafana dashboards are configured to monitor DAG run durations, success rates, failure rates, and task execution times.

6. Handling Changing Historical Data: Since historical files (e.g., today's data) can change frequently:

- **Streaming Phase:**
 - An event-driven or frequent polling approach triggers ingestion whenever a file changes.
 - Only the specific partition (**year/month/day**) is reloaded and overwritten in Iceberg.
- **Batch Phase (Daily Refresh):**
 - A scheduled DAG at 23:59 UTC processes the day's partition fully, ensuring late-arriving or corrected data is captured accurately.
 - dbt incremental models selectively refresh partitions, or a full-refresh can be triggered selectively for affected dates.

7. Incremental Processing Approach

- **Iceberg Tables:**
 - Partitions by **year/month/day** allow selective overwrite.
- **dbt Core:**
 - Models use **incremental** materializations.
 - **unique_key** based on date ensures idempotent operations.
 - Configured to **sync_all_columns** on schema change.

8. Data Catalog and Lineage

- **Cataloging**
 - **Trino catalog** definitions for both raw and aggregated Iceberg tables are maintained using Terraform.
 - Catalog metadata (columns, types, partitioning) is consistent and version-controlled.

- **Lineage**
 - **dbt** generates lineage graphs (**dbt docs generate**) that visualize upstream → downstream relationships.
 - **Airflow DAG views** help trace the pipeline stages and their dependencies.
 - Combined, they support auditability and troubleshooting.

8. Design Justifications:

Technology	Justification
S3	Durable, cost-effective object storage.
Iceberg	Supports ACID operations, schema evolution, partition evolution, and time-travel; ideal for evolving datasets.
Airflow	Mature orchestration framework supporting event-based and scheduled workflows, with native monitoring and alerting capabilities.
Lambda/Kubernetes	Cost-effective lightweight processing via Lambda, scalable long-running jobs via Kubernetes Pods.
dbt Core (Docker)	Clean, modular, and maintainable SQL-based transformation framework, isolated and reproducible in Docker containers.
Trino	Fast, scalable SQL engine optimized for object storage queries.
Slack	Real-time error notifications to engineering team.
Grafana	Visual dashboard monitoring for operational metrics.
Terraform	Ensures reproducibility and consistency when provisioning Trino catalogs and table structures, avoiding manual setup errors.

9. Why Data Lakehouse instead of Traditional Data Warehouse?

- The data arrives in semi-structured formats (CSV) and changes frequently, making rigid warehouse schemas impractical.
- Lakehouse architectures (with Iceberg) support schema evolution and ACID transactions at scale.
- Lower storage costs compared to traditional databases.
- Ability to perform both analytical and operational queries directly on object storage.
- Time-travel features in Iceberg allow easy rollback and historical analysis.

10. Possible Enhancements (for Full Production Setup)

- **Streaming ingestion** using S3 event notifications and Airflow triggers.
- **Data Catalog** integration (e.g., AWS Glue, Trino native catalog).
- **Data Lineage** visualization using dbt artifacts and Airflow DAG graphs.
- **Monitoring & Alerting** using Grafana dashboards and Slack integration.
- **Schema Evolution Management** with Iceberg automatic handling.
- **Automated Table Provisioning** with Terraform modules.
- **UpCloud Kubernetes** deployment for scalability.